实验 2：JavaScript 动态交互

实验类型：综合（4 学时）| 实验性质：必修 | 每组人数：1 人

一、实验目的

1. 掌握 JavaScript 基础语法（变量、函数、字符串、正则表达式）。
2. 深入理解 DOM 节点获取、内容与样式变更、classList 操作及事件绑定（blur、click）。
3. 掌握 BOM 对象应用（window.onload、定时器、confirm 确认框等）。
4. 掌握表单前端校验（区分失焦校验与按钮提交全量校验）。
5. 掌握浏览器 F12 控制台调试技巧。

二、实验环境

1. 操作系统：Windows 10/11 或 macOS
2. 开发工具：VS Code、Hbuilder 等前端编辑器
3. 浏览器：Google Chrome 或 Microsoft Edge

三、实验原理

1. **DOM 操作与事件驱动：**利用 DOM API 获取页面节点并操作 class/文字，通过绑定 onclick、onblur 事件响应用户交互。
2. **BOM 对象与定时器：**基于 window 对象，使用定时器实现动态时钟刷新，使用 confirm 弹出操作确认框。
3. **表单前端校验：**结合正则表达式对用户输入进行格式匹配，分离失焦（局部校验）与点击（全量提交）的逻辑。

四、实验内容与要求

1. 实验内容

以实验 1 完成的响应式个人简历页面（index.html + style.css）为基础，新增 JavaScript 脚本文件（script.js），升级实现以下三大交互功能：

（1）欢迎提示与动态时钟：页面加载时弹窗或控制台输出欢迎提示语；使用定时器更新时间（格式：YYYY-MM-DD HH:mm:ss）。

（2）主题切换交互：点击按钮切换 body 的 dark 类，动态改变界面配色与按钮文本。

（3）表单前端校验：表单元素失焦：仅校验并给出提示信息，不向后端提交数据；点击提交按钮：先校验且全过，confirm 确认框确认后再向后端提交数据。

- 姓名：非空，2-20 字符，仅支持中英文/·/空格，禁止数字；

- 邮箱：非空，标准邮箱正则格式（xxx@xxx.xx）；

- 留言：非空，5-500 字符；

2. 实验要求

（1）技术规范

- JS 代码写在外部 js/script.js 文件，在</body>标签前引入。

- HTML 结构复用实验 1，不改动原有页面布局。

- 代码缩进规范，添加必要注释。

- 控制台有明确的 console.log()调试信息输出。

（2） AI 工具使用规范

- 【允许】使用 AI 查询 API 用法（如 setTimeout、正则 test、classList 用法）或排查语法报错。

- 【禁止】严禁直接使用 AI 一键生成完整 JS 代码，验收环节将重点提问 DOM 操作、定时器、表单校验逻辑。

五、实验步骤与常见踩坑排查

5.1 实验步骤

【步骤 1：项目准备】

1. 复制实验 1 完整文件夹（保留原 HTML/CSS 结构）。

2. 新建 js/script.js，并在 index.html 的 </body>标签前引入：<script src="js/script.js"></script>

【步骤 2：欢迎提示与实时时钟】

1. 在 HTML 页脚或适当位置添加标签：。

2. 在 js/script.js 中编写初始化与动态时钟逻辑：

```
window.onload = function() {
    console.log("……");
    updateClock(); // 立即先渲染一次，避免页面加载后有 1 秒的空白延迟
    setInterval(updateClock, 1000); // 每隔 1 秒自动执行一次 updateClock
};
// 格式化时间：YYYY-MM-DD HH:mm:ss
function getFormatTime() {
    const now = new Date();
    const year = now.getFullYear();
    const month = String(now.getMonth() + 1).padStart(2, '0');
    const day = String(now.getDate()).padStart(2, '0');
    const hours = String(now.getHours()).padStart(2, '0');
    const minutes = String(now.getMinutes()).padStart(2, '0');
    const seconds = String(now.getSeconds()).padStart(2, '0');
    return `${year}-${month}-${day} ${hours}:${minutes}:${seconds}`;
```

```
}
// 更新 HTML 页面的时钟
function updateClock() {
    document.getElementById('clock').innerText = ……
}
```

【步骤 3：主题切换交互】

1. 在 HTML 中添加切换按钮：<button id="themeBtn">切换暗色主题</button>。

2. CSS 添加主题样式（后代选择器原理：只有 body 拥有 dark 类时，这组 CSS 才生效）。

```
body.dark { …… }
body.dark header { …… } /* 导航栏暗色 */
/* 自行扩展 header、卡片、表单、页脚等组件暗色样式 */
……
/* 补充主题按钮基础样式 */
……
```

3. JS 绑定点击事件与 class 切换。

```
// 获取按钮
const themeBtn = ……
// 绑定点击事件
themeBtn.onclick = function(){
    document.body.classList.toggle('dark');  //切换 body 的 dark 类：有就移除，没有就添加
    if(document.body.classList.contains('dark')){ //判断 body 是否包含 dark 类，修改按钮文字
        ……
    }else{
        ……
    }
}
```

【步骤 4：留言表单前端校验】

设计思路：checkForm()（失焦 onblur 触发，仅校验不提交）；checkSubmit()（按钮点击 onclick 触发，全量校验+确认提交）。

1. HTML 表单结构。

```
<form name="contactForm" action="#">
    <input type="text" id="nameInput" placeholder="姓名" onblur="checkForm()">
     ……
    <div id="formTip"></div>    <!--新增：用于校验提示 -->
    <button type="button" id="submitBtn" onclick="checkSubmit()">提交留言</button>
</form>
```

2. 编写 checkForm()函数。

```
function checkForm() {
    //正则表达式定义
    const nameReg = /^[\u4e00-\u9fa5a-zA-Z·\s]{2,20}$/;
    const emailReg = /^[a-zA-Z0-9_-]+(\.[a-zA-Z0-9_-]+)*@[a-zA-Z0-9_-]+(\.[a-zA-Z0-9_-]+)*\.[a-zA-Z]{2,}$/;
```

```
    //获取 DOM 元素
    const formTip = document.getElementById('formTip');
    const nameInput = document.getElementById('nameInput');
    ……
    //去除首尾空格
    const nameVal = nameInput.value.trim();
    ……
  //姓名判空
  if (nameVal === "") {
    formTip.className = "tip-error";
    formTip.innerText = "错误：姓名不能为空！";
    return false;
  }
  //3.姓名正则格式校验
  if (!nameReg.test(nameVal)) {
    formTip.className = "tip-error";
    formTip.innerText = "姓名仅支持中英文，不能包含数字，长度 2-20 字符";
    return false;
  }
  // 自行补充：邮箱校验、留言校验逻辑
  ……
  //全部校验通过
  formTip.className = "tip-success";
  formTip.innerText = "校验成功！";
  return true;
}
```

3. 编写 checkSubmit()提交函数。

```
function checkSubmit() {
  const flag = checkForm();
  if (!flag) {
    return false; //校验失败，不提交
  }
  //确认弹窗，用户确认才提交
  const send = confirm("是否要提交留言？");
  if (send) {
      ……  // 在 JS 中向后端提交数据
  }
}
```

4. 添加提示样式 CSS。

定义.tip-error（错误提示）、.tip-success（成功提示）样式，区分校验结果。

5.2 常见踩坑排查

1. JS 完全不生效：检查 script 引入位置，必须放在</body>之前；检查文件路径是否写错

（区分 script.js 和 js/script.js）。

2. getElementById 获取元素返回 null：DOM 还没有加载完成就执行 JS，脚本放在 body 末尾，或写在 window.onload 内部。
3. 失焦就提交数据：不要把提交逻辑写在 checkField()；checkField()仅做校验；只有 checkForm()才会调用 submit 提交。
4. blur 事件不触发：输入框没有真正失去焦点；检查 HTML 中 id、函数名大小写和 JS 保持一致（JS 区分大小写）。
5. 空格绕过校验：输入框取值必须调用.trim()去除首尾空格。
6. 点击提交按钮页面直接刷新：检查 HTML 中 <button> 标签是否添加了 type="button"，避免触发表单默认提交行为。

六、实验验收标准

检查项	权重	验收标准
动态时钟与控制台输出	25%	页面加载控制台打印欢迎文字；页脚时钟每秒自动刷新，时间格式 YYYY-MM-DD HH:mm:ss 正确。
主题切换交互	25%	按钮点击响应正常；亮色/暗色主题切换生效，按钮文字同步变更，控制台无报错。
留言表单前端校验	40%	blur 失焦只做校验，失焦不会提交表单；只有点击按钮才触发提交；姓名、邮箱、留言校验规则全部生效；错误红色提示、成功绿色提示；提交前弹出确认框。
代码规范及开发者工具调试技巧	10%	JS 代码注释清晰、缩进规范；复用实验 1 页面结构；可以解释 DOM、定时器、正则表单校验核心逻辑；能够使用 F12 做 console 查看日志或 Sources 断点调试。

七、实验报告要求

实验完成后，请撰写实验报告（纸质版），包含以下内容：

1. 实验目的（抄写或简述）。
2. 实验步骤：简述功能设计思路，粘贴关键 JavaScript 代码并添加注释。
3. 实验结果截图：
   (1) 亮色主题页面截图、暗色主题页面截图；
   (2) 动态时钟运行截图；
   (3) 表单校验：错误输入提示截图、合法输入校验成功截图；
   (4) 提交留言确认弹窗截图；
   (5) Chrome Console 控制台无报错截图。

4. 问题与解决方案：记录实验过程中的踩坑记录及排查思路（包括 AI 辅助排错的过程）。
5. 心得体会：总结 DOM、事件、表单交互开发体会。